TECHNICAL INTERVIEW GUIDE

The NLP Engineer's System Design Interview Guide

Preparation + 100 System Design Q&As: Scalability, LLMs, RAG, Transformers, and Production NLP Architecture

BONUS: TOP 35 MAANG NLP SYSTEM DESIGN

AUTHOR

Lamhot Siagian

LINKEDIN NEWSLETTER

AI Engineering Insider

System Design for NLP Interview Preparation

A Comprehensive Guide to Designing Scalable Natural Language Processing Systems

First Edition, 2026

Published independently by the author.

linkedin.com/in/lamhotsiagian

buymeacoffee.com/lamhot

For permissions and inquiries:
lamhotsiagian2025@gmail.com

About This Book

This book bridges two critical skill sets that top tech companies like Meta, Google, Amazon, and other FAANG companies expect from NLP engineers: deep knowledge of Natural Language Processing and the ability to design systems that scale to millions of users.

Each chapter covers a core NLP domain—from foundational mathematics through cutting-edge transformer architectures—and pairs it with 10 system design interview questions that test your ability to build production-grade NLP systems.

The questions are mapped to real-world system design challenges: scalability, high availability, data consistency, latency optimization, and more. Answers include architectural diagrams, Python code snippets, and the kind of trade-off analysis that interviewers expect.

Whether you are preparing for a senior ML engineer role or a staff-level NLP position, this book gives you the vocabulary, patterns, and confidence to tackle any system design question involving language technology.

Lamhot Siagian — AI Engineering Insider

linkedin.com/in/lamhotsiagian

0 Contents

linkedin.com/in/lamhotsiagian

1 Foundations of NLP & System Design 5

1.1 Chapter Overview 5

1.2 System Design Interview Questions 6

Q1: Design a Distributed Model Training Pipeline for a Large NLP Model 6
Q2: Design a Real-Time Feature Store for NLP Model Serving 8
Q3: Design an A/B Testing Framework for NLP Models 9
Q4: Design a GPU Resource Scheduler for NLP Workloads 10
Q5: Design a Model Registry and Versioning System 11
Q6: Design a Data Pipeline for Training Corpus Management 13
Q7: Design a Monitoring System for NLP Model Performance Drift 14
Q8: Design a Hyperparameter Optimization Service 15
Q9: Design a Secure Multi-Tenant NLP Platform 17
Q10: Design an Automated ML Pipeline for Continuous NLP Model Retraining 18

2 Text Preprocessing at Scale 20

2.1 Chapter Overview 20

2.2 System Design Interview Questions 21

Q1: Design a Real-Time Text Preprocessing Service for 1M Requests/Second 21
Q2: Design a Distributed Tokenizer Training Pipeline 22
Q3: Design a PII Detection and Redaction Pipeline 24
Q4: Design a Language Detection Service for Multilingual Content 25
Q5: Design a Text Normalization Pipeline for Search 26
Q6: Design a Streaming Text Cleaning Pipeline for Social Media Data 28
Q7: Design a Text Encoding and Character Set Management System 30
Q8: Design a Regex-Based Rule Engine for Text Classification 31

Q9: Design a Data Annotation Platform for NLP Tasks 33
Q10: Design a Multi-Stage Text Preprocessing Pipeline with Data Lineage 34

3 Text Representation & Embeddings at Scale 37

3.1 Chapter Overview 37
3.2 System Design Interview Questions 38
Q1: Design a Large-Scale Embedding Serving Infrastructure 38
Q2: Design a Vector Similarity Search System 39
Q3: Design a TF-IDF Based Search Engine for 100M Documents 41
Q4: Design an Embedding Model Update Pipeline 42
Q5: Design a Multi-Modal Embedding System (Text + Images) 44
Q6: Design an N-gram Index for Autocomplete and Spell Correction 45
Q7: Design a Word Embedding Training Infrastructure 47
Q8: Design an Embedding Compression System for Mobile Deployment 48
Q9: Design a Dynamic Embedding Update System for Evolving Vocabulary 49
Q10: Design a Hybrid Search System Combining Sparse and Dense Retrieval 51

4 Core NLP Tasks in Production 54

4.1 Chapter Overview 54
4.2 System Design Interview Questions 55
Q1: Design a Real-Time NER Service for Content Understanding 55
Q2: Design a Knowledge Graph Construction Pipeline Using NLP 56
Q3: Design a Multi-Language NER System 58
Q4: Design a Dependency Parsing Service for Information Extraction 59
Q5: Design a Coreference Resolution System for Document Understanding 61
Q6: Design a Sentence Segmentation Service for Multilingual Text 62
Q7: Design an Entity Linking System at Web Scale 64
Q8: Design an NLP Pipeline Orchestrator with Error Recovery 65
Q9: Design an Active Learning System for NER Model Improvement 67
Q10: Design a Constituency Parsing System with Parse Caching 69

5 Classical Machine Learning for NLP 71

5.1 Chapter Overview 71
5.2 System Design Interview Questions 72
Q1: Design a Spam Classification System Using Naive Bayes at Scale 72
Q2: Design a Text Classification System with Logistic Regression 73
Q3: Design an SVM-Based Document Similarity System 75
Q4: Design a CRF-Based NER System with Online Learning 76
Q5: Design an HMM-Based POS Tagger with Caching 78

Q6: Design a Sentiment Analysis System Using Ensemble Classical Models 79
Q7: Design a Feature Engineering Pipeline for Classical NLP Models 81
Q8: Design a Multi-Label Classification System for Content Tagging 83
Q9: Design a Classical ML Model Serving Infrastructure 84
Q10: Design a Feedback Loop System for Continuous Model Improvement 86

6 Deep Learning for NLP at Scale 88

6.1 Chapter Overview 88
6.2 System Design Interview Questions 89
Q1: Design a Real-Time LSTM-Based Sequence Prediction Service 89
Q2: Design a GPU Cluster for Deep Learning NLP Training 90
Q3: Design a CNN-Based Text Classification System for Content Moderation 92
Q4: Design an Attention-Based Model Serving System with Dynamic Batching 93
Q5: Design a Bidirectional LSTM Pipeline for Sequence Labeling 95
Q6: Design a Model Distillation Pipeline for Edge Deployment 97
Q7: Design a GRU-Based Streaming NLP System 98
Q8: Design a Multi-Task Deep Learning NLP System 99
Q9: Design a Model Versioning and Rollback System for Deep Learning 101
Q10: Design a Transfer Learning Infrastructure for Domain Adaptation 102

7 Transformers & Large Language Models 105

7.1 Chapter Overview 105
7.2 System Design Interview Questions 106
Q1: Design an LLM Inference Infrastructure for 10,000 Concurrent Users 106
Q2: Design a RAG (Retrieval-Augmented Generation) System for Enterprise Search 108
Q3: Design a Fine-Tuning Platform for Domain-Specific LLMs 110
Q4: Design a Prompt Management and Caching System for LLM APIs 111
Q5: Design a BERT-Based Search and Ranking System 113
Q6: Design a Multi-Tenant LLM API Gateway 115
Q7: Design a Speculative Decoding System for LLM Acceleration 116
Q8: Design a Transformer Model Quantization and Optimization Pipeline 118
Q9: Design a Transformer-Based Document Processing Pipeline 120
Q10: Design an LLM Evaluation and Safety Testing Framework 122

8 Key NLP Application Areas 124

8.1 Chapter Overview 124
8.2 System Design Interview Questions 125
Q1: Design a Real-Time Sentiment Analysis Platform for Social Media Monitoring 125
Q2: Design a Machine Translation Service Supporting 100 Language Pairs 127

Q3: Design a Scalable Text Summarization Service 128
Q4: Design a Production Question Answering System 130
Q5: Design a Dialogue System Infrastructure for a High-Traffic Chatbot 132
Q6: Design a Topic Modeling System for Content Discovery 134
Q7: Design a Document Information Extraction Pipeline 135
Q8: Design a Personalized Text Generation System 137
Q9: Design a Real-Time Content Moderation System 138
Q10: Design a Multilingual Chatbot with Intent Disambiguation 140

9 Advanced Topics in NLP Systems 143

9.1 Chapter Overview 143
9.2 System Design Interview Questions 144
Q1: Design a Few-Shot Learning Platform for Rapid NLP Task Deployment 144
Q2: Design a Streaming Speech-to-Text System 146
Q3: Design a Knowledge Graph Serving System for NLP Applications 147
Q4: Design an NLP Model Evaluation Pipeline with Statistical Rigor 149
Q5: Design a Zero-Shot Classification API 151
Q6: Design a Relation Extraction Pipeline for Document Intelligence 153
Q7: Design a Multilingual NLP System with Cross-Lingual Transfer 155
Q8: Design a Neural Text-to-Speech System 157
Q9: Design a Bias Detection and Mitigation System for NLP Models 158
Q10: Design a Continuous Evaluation System for Production NLP 160

10 Practical & Ethical Considerations 163

10.1 Chapter Overview 163
10.2 System Design Interview Questions 164
Q1: Design a Responsible AI Governance System for NLP Models 164
Q2: Design a Privacy-Preserving NLP Training System 166
Q3: Design a Dataset Curation and Quality Control System 168
Q4: Design a Human-in-the-Loop NLP System for High-Stakes Decisions 170
Q5: Design an Explainable NLP System for Regulated Industries 172
Q6: Design a Scalable Model Serving System Using Hugging Face Inference Endpoints 174
Q7: Design a Fairness-Aware NLP Model Training Pipeline 176
Q8: Design an NLP System for Regulatory Compliance Monitoring 178
Q9: Design a Responsible Data Deletion System for NLP Models 180
Q10: Design a Responsible AI Monitoring Dashboard for NLP Systems 182

11 Bonus: Top 35 MAANG NLP System Design Questions 185

11.1 Google — Top 7 NLP System Design Questions 186

G-Q1: Design Google Search's Query Understanding Pipeline (MUM/BERT at Scale) 186
G-Q2: Design Google Translate's Neural Machine Translation Serving System 188
G-Q3: Design a Real-Time Content Safety System for YouTube Using NLP 189
G-Q4: Design Google Assistant's Natural Language Understanding Service 190
G-Q5: Design a Semantic Search System for Google Workspace (Docs, Drive, Gmail) 192
G-Q6: Design an AutoComplete and Query Suggestion System for Google Search 193
G-Q7: Design Google's Large-Scale Multilingual Document Classification System 194

11.2 Meta — Top 7 NLP System Design Questions 196

M-Q1: Design Meta's News Feed Ranking NLP Signal Extraction System 196
M-Q2: Design Meta's Multilingual Content Moderation NLP System 197
M-Q3: Design WhatsApp's On-Device NLP for Smart Reply Suggestions 199
M-Q4: Design Meta AI's Retrieval-Augmented Generation System for Llama Integration 200
M-Q5: Design Instagram's Hashtag and Caption Understanding System 202
M-Q6: Design Meta's Automated Translation System for Cross-Language Social Interaction 203
M-Q7: Design a Real-Time Hate Speech Detection System for Live Video on Facebook 204

11.3 Amazon — Top 7 NLP System Design Questions 206

A-Q1: Design Alexa's Intent Recognition and Slot Filling System at Scale 206
A-Q2: Design Amazon Product Search's Semantic Understanding Layer 207
A-Q3: Design AWS Comprehend's Multi-Tenant NLP API 209
A-Q4: Design Amazon Review Sentiment Aggregation System 210
A-Q5: Design Amazon Lex's Conversational AI Backend 211
A-Q6: Design Amazon's Query Rewriting System for Sponsored Product Search 213
A-Q7: Design an NLP System for Amazon Customer Service Email Triage 214

11.4 Apple — Top 7 NLP System Design Questions 216

Ap-Q1: Design Siri's On-Device Natural Language Understanding System 217
Ap-Q2: Design Apple's Mail Smart Reply System with On-Device Privacy 218
Ap-Q3: Design Spotlight Search's Natural Language Query System 220
Ap-Q4: Design Apple's Autocorrect and Predictive Text System 221
Ap-Q5: Design Apple's On-Device Federated Learning System for Siri Personalization 223
Ap-Q6: Design Apple's Document Intelligence System for iOS Files App 224
Ap-Q7: Design Apple's Private Cloud Compute NLP Inference Architecture 226

11.5 Netflix — Top 7 NLP System Design Questions 227

N-Q1: Design Netflix's Multilingual Subtitle Generation and Quality Pipeline 228
N-Q2: Design Netflix's Content Tagging NLP System for Recommendations 229
N-Q3: Design Netflix Search's Semantic Understanding for Mood-Based Queries 231
N-Q4: Design a Personalized Synopsis Generation System for Netflix 232
N-Q5: Design Netflix's Dialogue-Based Content Safety Review System 234
N-Q6: Design a Cross-Lingual NLP System for Netflix's Global Content Strategy 236

N-Q7: Design Netflix's A/B Testing Framework for NLP-Driven Recommendation Changes 237

Final Words **241**

1 Foundations of NLP & System Design

linkedin.com/in/lamhotsiagian

Before building production NLP systems, you need a solid command of the mathematical and computational building blocks. This chapter covers the foundational concepts—linear algebra, probability, statistics, and core machine learning—that underpin every NLP system at scale.

1.1 Chapter Overview

Every NLP system at a company like Meta or Google is built on a stack of mathematical primitives. When an interviewer asks you to design a real-time content moderation system, they expect you to understand not just the ML model, but how linear algebra operations map to GPU compute, how probability distributions inform your sampling strategy, and how gradient descent interacts with distributed training across hundreds of machines.

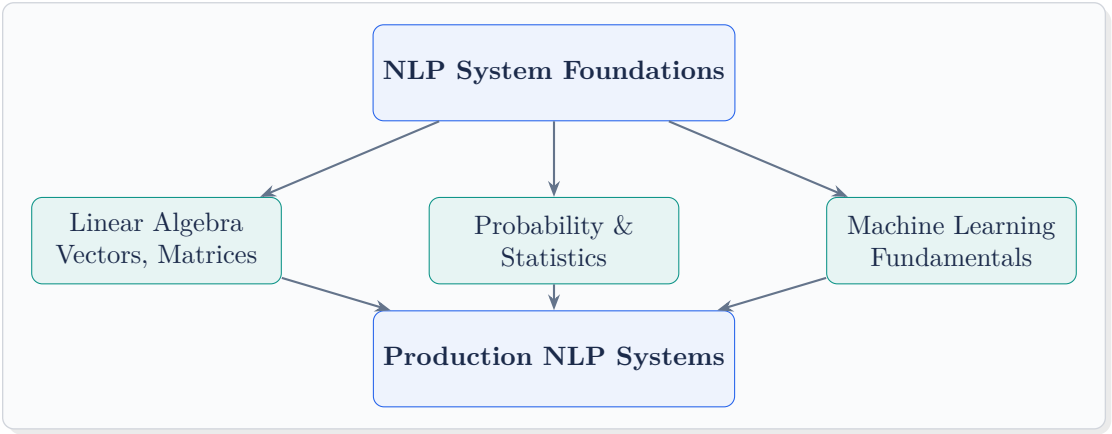
The foundations break down into three pillars:

Linear Algebra powers the core computations. Word embeddings are vectors, attention matrices are matrix multiplications, and dimensionality reduction techniques like PCA and SVD are used to compress representations for efficient storage and retrieval.

Probability & Statistics drives everything from language modeling (predicting the next token is fundamentally a probability distribution over vocabulary) to evaluation metrics, A/B testing your deployed system, and Bayesian approaches to uncertainty estimation.

Machine Learning Fundamentals tie it all together—supervised and unsupervised learning paradigms, optimization via gradient descent, regularization to prevent overfit-

ting, and the bias-variance trade-off that determines whether your model generalizes in production.



Key Insight

At FAANG-level interviews, foundation questions are never asked in isolation. You will be expected to connect math to system decisions: "Why use float16 instead of float32?" is a linear algebra question, a latency question, and a cost management question all at once.

1.2 System Design Interview Questions

Q1: Design a Distributed Model Training Pipeline for a Large NLP Model

System Design Focus: Scalability Cost Management

How would you design a distributed training system capable of training a 10-billion parameter language model across a cluster of GPUs? Walk through the architecture, data parallelism vs. model parallelism, and how you would handle gradient synchronization.

Architecture Overview:

The system requires a multi-node GPU cluster orchestrated by a training coordinator. The key architectural decisions involve choosing between data parallelism (replicate the model across GPUs, split the data) and model parallelism (split the

model across GPUs), or a hybrid approach.

Core Components:

- **Training Orchestrator:** Manages job scheduling, checkpoint management, and fault recovery. Use Kubernetes with custom operators for GPU-aware scheduling.
- **Data Pipeline:** Distributed data loading with prefetching. Store training data in object storage (S3), use a streaming dataloader to avoid memory bottlenecks.
- **Gradient Synchronization:** Use AllReduce (via NCCL) for data parallelism. For 10B parameters, use ZeRO-3 optimization to partition optimizer states, gradients, and parameters across GPUs.
- **Checkpointing Service:** Async checkpointing every N steps to distributed storage, enabling recovery without full restart.

```python
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as
    DDP

def setup_distributed(rank, world_size):
    dist.init_process_group("nccl", rank=rank,
                            world_size=world_size)
    torch.cuda.set_device(rank)

def train_step(model, optimizer, dataloader):
    model = DDP(model, device_ids=[local_rank])
    for batch in dataloader:
        outputs = model(batch["input_ids"].cuda())
        loss = outputs.loss
        loss.backward()  # Gradients synced via AllReduce
        optimizer.step()
        optimizer.zero_grad()
```

Scalability: Horizontal scaling by adding GPU nodes. Use mixed-precision training (float16) to halve memory and double throughput. Gradient accumulation lets you simulate larger batch sizes without more GPUs.

Cost Management: Use spot/preemptible instances for non-critical training runs. Implement elastic training that can scale down gracefully when spot instances are reclaimed.

Q2: Design a Real-Time Feature Store for NLP Model Serving

System Design Focus: **Latency & Performance** **Data Storage**
Design a feature store that serves precomputed NLP features (embeddings, TF-IDF vectors, entity tags) to ML models in production with sub-10ms latency at 100K QPS.

Architecture:
A two-tier storage architecture separates *offline* feature computation from *online* serving:
Offline Layer: Batch pipelines (Spark/Beam) compute features from raw text and write to a feature registry. Features are versioned and stored in a columnar format (Parquet on S3).
Online Layer: A low-latency serving layer backed by Redis Cluster for hot features and a fallback to DynamoDB for warm features. A consistency daemon syncs offline computations to the online store.
Key Design Decisions:

- **Caching:** L1 cache (in-process LRU, 1ms) and L2 cache (Redis, 3-5ms). Cache hit rate target: ¿95%.
- **Sharding:** Consistent hashing on entity ID across Redis nodes. Each shard handles 20K QPS.
- **Serialization:** Protocol Buffers for compact encoding of embedding vectors. A 768-dim float32 embedding compresses to 3KB with quantization.

```
import redis
import numpy as np
from typing import Optional

class NLPFeatureStore:
    def __init__(self, redis_cluster_nodes):
        self.redis = redis.RedisCluster(
            startup_nodes=redis_cluster_nodes,
            decode_responses=False
        )
        self.local_cache = {}  # LRU cache

    def get_embedding(self, entity_id: str) ->
        Optional[np.ndarray]:
        # L1: in-process cache
        if entity_id in self.local_cache:
```

```
            return self.local_cache[entity_id]
        # L2: Redis cluster
        data = self.redis.get(f"emb:{entity_id}")
        if data:
            embedding = np.frombuffer(data,
                dtype=np.float32)
            self.local_cache[entity_id] = embedding
            return embedding
        return None  # Cache miss -> fallback to DB
```

Performance: P99 latency ¡ 8ms at 100K QPS. Redis Cluster with 6 shards provides sufficient throughput with 2x replication for fault tolerance.

Q3: Design an A/B Testing Framework for NLP Models

System Design Focus: **Data Consistency** **Observability**
How would you design an A/B testing platform that allows data scientists to safely experiment with different NLP models in production while maintaining statistical rigor?

Architecture:
The framework consists of four services: an *Experiment Manager* (defines experiments), a *Traffic Router* (assigns users to variants), a *Metrics Collector* (captures outcomes), and a *Statistical Engine* (analyzes results).
Traffic Assignment: Use deterministic hashing (MurmurHash3 of user ID + experiment ID) to ensure consistent assignment. A user always sees the same variant, even across sessions. This avoids the consistency problem of random assignment.
Guardrails:

- Automatic rollback if error rate exceeds baseline by ¿2%.
- Gradual ramp-up: 1% → 5% → 25% → 50%.
- Isolation between experiments using traffic slicing to avoid interaction effects.

Statistical Rigor: Pre-register hypotheses and sample sizes. Use sequential testing (not fixed-horizon) to allow early stopping. Apply Bonferroni correction for multiple comparisons.

```
import hashlib
```

```python
from scipy import stats

def assign_variant(user_id: str, experiment_id: str,
                   num_variants: int = 2) -> int:
    hash_input = f"{user_id}:{experiment_id}"
    hash_val = int(hashlib.md5(
        hash_input.encode()).hexdigest(), 16)
    return hash_val % num_variants

def check_significance(control, treatment, alpha=0.05):
    t_stat, p_value = stats.ttest_ind(control, treatment)
    return {
        "significant": p_value < alpha,
        "p_value": round(p_value, 4),
        "effect_size": (np.mean(treatment) -
           np.mean(control))
                       / np.std(control)
    }
```

Observability: Every experiment logs assignment events, model predictions, and user interactions to a unified event stream (Kafka). A real-time dashboard shows key metrics per variant with confidence intervals.

Q4: Design a GPU Resource Scheduler for NLP Workloads

System Design Focus: Cost Management Scalability
Design a system that efficiently schedules NLP training and inference jobs across a shared GPU cluster, optimizing for utilization and cost.

Architecture:
Build a priority-based scheduler with three queues: *Critical* (production inference), *Standard* (scheduled training), and *Best-Effort* (experiments, fine-tuning). The scheduler implements bin-packing to maximize GPU utilization.
Key Components:

- **Resource Broker:** Tracks GPU memory, compute utilization, and network bandwidth across all nodes. Maintains a real-time resource graph.
- **Job Profiler:** Estimates resource requirements based on model architecture. A 1B parameter model in float16 needs 2GB GPU memory for inference; training

needs 6x that.

- **Preemption Manager:** Best-effort jobs can be preempted (with checkpoint) when critical jobs arrive. Preempted jobs resume automatically when resources free up.
- **Auto-Scaler:** Monitors queue depth and scales the cluster using cloud APIs. Spot instances for best-effort, on-demand for critical.

```python
class GPUScheduler:
    def __init__(self):
        self.queues = {
            "critical": PriorityQueue(),
            "standard": PriorityQueue(),
            "best_effort": PriorityQueue()
        }
        self.gpu_nodes = {}  # node_id -> GPUNode

    def schedule(self, job):
        required_gpus = self.estimate_resources(job)
        available =
           self.find_available_nodes(required_gpus)
        if available:
            self.assign(job, available)
        elif job.priority == "critical":
            self.preempt_and_assign(job, required_gpus)
        else:
            self.queues[job.priority].put(job)

    def estimate_resources(self, job):
        param_bytes = job.model_params * 2  # float16
        if job.type == "training":
            return param_bytes * 6  # gradients +
               optimizer
        return param_bytes  # inference only
```

Cost Optimization: GPU utilization target ¿ 80%. Multi-tenancy with memory isolation using NVIDIA MPS. Time-of-day pricing awareness: schedule large training jobs during off-peak hours.

Q5: Design a Model Registry and Versioning System

System Design Focus: **Data Consistency** **High Availability**
Design a model registry that tracks all NLP model versions, their lineage, performance metrics, and deployment status across the organization.

Architecture:
The model registry is a metadata service backed by PostgreSQL for structured metadata (versions, metrics, lineage) and object storage (S3) for model artifacts. It provides a REST API and CLI for model lifecycle management.
Data Model:

- **Model:** name, owner, description, task type
- **Version:** model ID, version number, artifact URI, training config, metrics (accuracy, F1, latency), stage (dev/staging/prod)
- **Lineage:** parent model, training dataset hash, preprocessing pipeline version, hyperparameters

Consistency Guarantees: Stage transitions (dev → staging → prod) use database transactions with optimistic locking. Only one version can be in "prod" stage per model at any time. Promotion requires passing automated validation gates.
High Availability: PostgreSQL with streaming replication (primary + 2 read replicas). Model artifacts on S3 with cross-region replication. The API layer runs behind a load balancer with 3 instances.

```
class ModelRegistry:
    def register_model(self, name, artifact_path,
        metrics):
         version = self.get_next_version(name)
         artifact_uri = self.upload_artifact(artifact_path)
         # Atomic registration with lineage
         with self.db.transaction():
             model_version = ModelVersion(
                 model_name=name,
                 version=version,
                 artifact_uri=artifact_uri,
                 metrics=metrics,
                 stage="development",
                 created_at=datetime.utcnow()
             )
             self.db.insert(model_version)
```

```
        return model_version

    def promote(self, name, version, target_stage):
        with self.db.transaction():
            # Demote current prod version
            self.db.update(name, stage=target_stage,
                           set_stage="archived")
            # Promote new version
            self.db.update(name, version=version,
                           set_stage=target_stage)
```

Q6: Design a Data Pipeline for Training Corpus Management

System Design Focus: **Data Storage & Retrieval** **Scalability**
Design a system that ingests, deduplicates, cleans, and versions terabyte-scale text corpora used for training NLP models.

Architecture:
A multi-stage pipeline: **Ingestion** → **Deduplication** → **Cleaning** → **Quality Scoring** → **Versioned Storage**.
Ingestion Layer: Web crawlers, API connectors, and file importers push raw documents into a staging area (S3 raw bucket). Each document gets a UUID and metadata envelope (source, timestamp, language, content hash).
Deduplication: Two-phase approach. First, exact dedup using SHA-256 content hashes. Second, near-duplicate detection using MinHash LSH (Locality-Sensitive Hashing) with Jaccard similarity threshold of 0.8. This runs as a distributed Spark job.
Cleaning Pipeline: Language detection (fastText lid), encoding normalization, HTML/boilerplate removal, PII redaction (regex + NER-based), and toxicity filtering. Each stage is a separate microservice connected via Kafka topics.
Versioning: Use DVC (Data Version Control) or Delta Lake for dataset versioning. Each training run references an immutable dataset snapshot by version hash.

```
from datasketch import MinHash, MinHashLSH

def build_minhash(text, num_perm=128):
    mh = MinHash(num_perm=num_perm)
```

```
    for word in text.lower().split():
        mh.update(word.encode('utf-8'))
    return mh

def deduplicate_corpus(documents, threshold=0.8):
    lsh = MinHashLSH(threshold=threshold, num_perm=128)
    unique_docs = []
    for doc_id, text in documents:
        mh = build_minhash(text)
        if not lsh.query(mh):  # No near-duplicates found
            lsh.insert(doc_id, mh)
            unique_docs.append((doc_id, text))
    return unique_docs
```

Scale: Processing 10TB of text requires 500 Spark executors with MinHash. The dedup index fits in memory with 128-bit signatures per document (16 bytes per doc for 1B documents = 16GB).

Q7: Design a Monitoring System for NLP Model Performance Drift

System Design Focus: **Observability** **High Availability**
Design a monitoring system that detects when deployed NLP models begin to degrade in production due to data drift, concept drift, or infrastructure issues.

Architecture:
Three monitoring layers: **Infrastructure Metrics** (latency, throughput, errors), **Data Distribution Monitoring** (input drift detection), and **Model Quality Monitoring** (output quality tracking).
Data Drift Detection: Sample incoming requests and compare feature distributions against a reference window (the validation set from training). Use Population Stability Index (PSI) for numerical features and Jensen-Shannon divergence for embedding distributions.
Concept Drift Detection: Track model confidence distributions over time. A shift in the confidence histogram signals that the relationship between inputs and outputs has changed. Use ADWIN (Adaptive Windowing) for streaming drift detection.
Alert System: Three severity levels. *Warning* (PSI ¿ 0.1): investigate within

24h. *Critical* (PSI ¿ 0.25 or error rate spike): page on-call. *Emergency* (model returning errors): automatic fallback to previous version.

```python
import numpy as np
from scipy.spatial.distance import jensenshannon

class DriftDetector:
    def __init__(self, reference_distribution):
        self.reference = reference_distribution

    def compute_psi(self, current, bins=10):
        ref_hist, edges = np.histogram(
            self.reference, bins=bins, density=True)
        curr_hist, _ = np.histogram(
            current, bins=edges, density=True)
        # Avoid division by zero
        ref_hist = np.clip(ref_hist, 1e-6, None)
        curr_hist = np.clip(curr_hist, 1e-6, None)
        psi = np.sum(
            (curr_hist - ref_hist) * np.log(
                curr_hist / ref_hist))
        return psi

    def check_embedding_drift(self, current_embeddings):
        js_div = jensenshannon(
            self.reference.mean(axis=0),
            current_embeddings.mean(axis=0))
        return {"js_divergence": js_div,
                "drifted": js_div > 0.15}
```

High Availability: The monitoring system itself must be highly available. Run as a stateless service behind a load balancer. Metric storage in a time-series database (InfluxDB/Prometheus) with 30-day retention and downsampled long-term storage.

Q8: Design a Hyperparameter Optimization Service

System Design Focus: Scalability Cost Management
Design a scalable service that automatically tunes hyperparameters for NLP models, managing hundreds of concurrent experiments efficiently.

Architecture:
A central **Optimization Server** coordinates trials. It maintains a search space definition, selects the next hyperparameter configuration using Bayesian optimization (Tree-structured Parzen Estimators or Gaussian Processes), and dispatches trial jobs to the GPU cluster.

Key Design Decisions:

- **Search Strategy:** Start with random search for initial exploration (first 20 trials), then switch to Bayesian optimization. For NLP, key hyperparameters include learning rate, batch size, warmup steps, dropout rate, and weight decay.
- **Early Stopping:** Use Successive Halving or Hyperband to terminate underperforming trials early. Evaluate at 10%, 25%, 50%, 100% of epochs. This reduces compute costs by 3-5x.
- **Trial Management:** Each trial is a Kubernetes job. Trials report intermediate metrics via gRPC. Failed trials are retried once, then marked as failed.
- **Results Database:** All trial configurations and results are stored in PostgreSQL for analysis and reproducibility.

```python
import optuna

def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    batch_size = trial.suggest_categorical(
        "batch_size", [16, 32, 64])
    warmup_ratio = trial.suggest_float(
        "warmup_ratio", 0.0, 0.2)
    dropout = trial.suggest_float("dropout", 0.0, 0.5)

    model = train_nlp_model(
        lr=lr, batch_size=batch_size,
        warmup_ratio=warmup_ratio, dropout=dropout
    )
    f1_score = evaluate(model, val_dataset)
    return f1_score

# Distributed optimization across workers
study = optuna.create_study(
    direction="maximize",
    storage="postgresql://optuna:pw@db/optuna",
    study_name="bert_ner_tuning",
    pruner=optuna.pruners.HyperbandPruner()
)
```

```
study.optimize(objective, n_trials=200, n_jobs=10)
```

Cost: A 200-trial HPO run for a BERT-base model costs approximately $500-$1000 on cloud GPUs. With Hyperband pruning, this drops to $150-$300.

Q9: Design a Secure Multi-Tenant NLP Platform

System Design Focus: **Security** **High Availability**
Design an NLP-as-a-Service platform where multiple teams within an organization can train and deploy models with proper isolation, access control, and data security.

Architecture:
A multi-tenant platform with namespace isolation per team. Each team gets a logical workspace with its own resource quotas, model registry, and data store, running on shared infrastructure.
Security Layers:

- **Authentication:** OAuth 2.0 / OIDC integration with corporate IdP. Service-to-service auth via mTLS with short-lived certificates.
- **Authorization:** RBAC with roles: Admin, ML Engineer, Data Scientist, Viewer. Fine-grained permissions on models, datasets, and endpoints.
- **Data Isolation:** Each tenant's training data is encrypted at rest with tenant-specific KMS keys. Network policies enforce namespace isolation in Kubernetes.
- **Audit Logging:** Every API call, model access, and data operation is logged to an immutable audit trail.

```
from functools import wraps

def require_permission(resource_type, action):
    def decorator(func):
        @wraps(func)
        def wrapper(request, *args, **kwargs):
            user = authenticate(request.token)
            tenant = resolve_tenant(request)
            if not authorize(user, tenant,
                             resource_type, action):
                raise PermissionDenied(
                    f"User {user.id} cannot {action} "
                    f"{resource_type} in {tenant.name}")
```

```
            audit_log(user, tenant, resource_type, action)
            return func(request, *args, **kwargs)
        return wrapper
    return decorator

@require_permission("model", "deploy")
def deploy_model(request, model_id, version):
    # Only executes if user has deploy permission
    endpoint = serving_service.deploy(
        model_id, version,
        namespace=request.tenant.namespace)
    return endpoint
```

High Availability: Control plane runs across 3 AZs. Each tenant's inference endpoints have independent auto-scaling policies. Blast radius is limited: one tenant's outage does not affect others.

Q10: Design an Automated ML Pipeline for Continuous NLP Model Retraining

System Design Focus: Communication Between Services Observability
Design an end-to-end MLOps pipeline that automatically retrains NLP models when performance degrades or new data becomes available.

Architecture:
An event-driven pipeline with five stages: **Trigger** → **Data Preparation** → **Training** → **Validation** → **Deployment**. Each stage is a separate service communicating through an event bus (Kafka).
Trigger Conditions:

- **Performance-based:** Drift detector signals model degradation (PSI ¿ 0.2).
- **Data-based:** New labeled data exceeds a threshold (e.g., 10K new samples).
- **Scheduled:** Weekly retraining regardless of triggers.

Pipeline Orchestration: Use Apache Airflow or Kubeflow Pipelines. Each step is idempotent and retryable. The pipeline publishes events at each stage transition, enabling downstream services (monitoring, notifications) to react.
Validation Gates: Before deployment, the new model must beat the current production model on a held-out test set by a statistically significant margin.

Shadow deployment (running both models on live traffic) provides additional confidence.

```
# Airflow DAG for NLP retraining pipeline
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import timedelta

dag = DAG("nlp_retrain",
          schedule_interval=timedelta(weeks=1),
          catchup=False)

prepare = PythonOperator(
    task_id="prepare_data",
    python_callable=prepare_training_data,
    dag=dag)

train = PythonOperator(
    task_id="train_model",
    python_callable=train_model,
    dag=dag)

validate = PythonOperator(
    task_id="validate_model",
    python_callable=run_validation_suite,
    dag=dag)

deploy = PythonOperator(
    task_id="deploy_canary",
    python_callable=canary_deploy,
    dag=dag)

prepare >> train >> validate >> deploy
```

Communication: Kafka topics per stage (data.prepared, model.trained, model.validated, model.deployed). Services consume events asynchronously. Dead letter queues capture failed messages for debugging.
Observability: Distributed tracing (Jaeger) across the full pipeline. Each run produces a lineage graph showing data version → model version → deployment version.

2 Text Preprocessing at Scale

linkedin.com/in/lamhotsiagian

Raw text is messy. Before any NLP model can work with language, the text must be cleaned, normalized, and transformed into a consistent format. At production scale, preprocessing is not a one-off script—it is a distributed pipeline that must handle billions of documents reliably, consistently, and efficiently.

2.1 Chapter Overview

Text preprocessing is the unsung hero of NLP systems. At FAANG companies processing billions of text documents daily—user posts, search queries, customer support tickets, comments—the preprocessing pipeline is often the most critical (and most fragile) component.

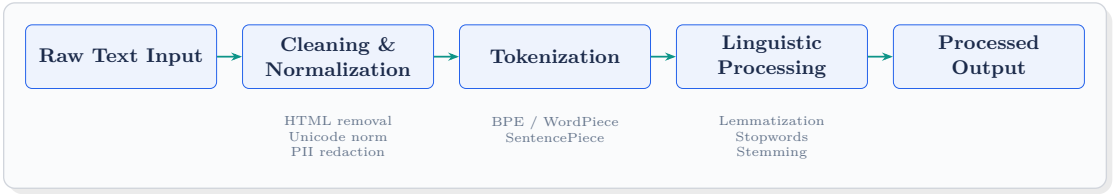
The core preprocessing operations form a standard pipeline:

Tokenization is the foundation—splitting text into meaningful units (words, subwords, or characters). Modern NLP overwhelmingly uses subword tokenization (BPE, WordPiece, SentencePiece) because it handles out-of-vocabulary words gracefully and keeps vocabulary sizes manageable.

Normalization standardizes text: lowercasing, Unicode normalization (NFC/NFKC), accent removal, and handling special characters. The choice of normalization depends heavily on the downstream task—sentiment analysis may want to preserve case ("AMAZING" signals intensity), while search needs case-insensitive matching.

Cleaning removes noise: HTML tags, URLs, email addresses, excessive whitespace, and encoding artifacts. PII (Personally Identifiable Information) redaction is increasingly a legal requirement, not just a best practice.

Linguistic Processing includes stemming (crude suffix stripping), lemmatization (reducing to dictionary form using morphological analysis), stopword removal, and part-of-speech-aware filtering.



Production Reality

At scale, preprocessing consistency is paramount. If your training pipeline lowercases text but your inference pipeline does not, your model will silently degrade. Version your preprocessing logic alongside your models.

2.2 System Design Interview Questions

Q1: Design a Real-Time Text Preprocessing Service for 1M Requests/Second

System Design Focus: Scalability Latency & Performance
Design a preprocessing service that normalizes, tokenizes, and cleans user-generated text in real time at 1 million requests per second with P99 latency under 20ms.

Architecture:
A stateless preprocessing service deployed as a horizontally scaled fleet behind a load balancer. Each instance runs preprocessing in-process with no external dependencies on the hot path.
Key Design:

- **Stateless Workers:** Each pod handles 50K RPS. Need 20 pods with headroom. Auto-scale based on CPU utilization (target 60%).
- **In-Memory Tokenizer:** Load the tokenizer vocabulary (BPE merge table) into memory at startup. Tokenization is CPU-bound, not I/O-bound, so no need for external calls.
- **Pipeline Optimization:** Fuse operations—normalize and tokenize in a single pass rather than multiple string copies. Use Rust/C++ for the hot path with